✅ Problem Statement 5

Set up Jenkins to trigger automated builds when code is pushed to GitHub

---

🎯 What You Have Done (Simple Meaning)

👉 You created an automation system where:

- **Code is stored on GitHub**
- **Jenkins monitors it**
- **Whenever code is updated → Jenkins automatically runs your Python program**

👉 This is called CI (Continuous Integration)

🔄 Step-by-Step Explanation (What Each Step Does)

🔹 1. Start Jenkins

sudo systemctl start jenkins.service

👉 Starts Jenkins server

🔹 2. Open in Browser

http://localhost:8080

👉 Jenkins UI opens
👉 Enter admin password (given in terminal)

🔹 3. Create New Job

- Click New Item
- Enter name
- Select Freestyle Project

👉 This job will run your code automatically

🔹 4. Connect GitHub Repository

- Select Git
- Paste repository URL

👉 Jenkins now knows where your code is stored

🔹 5. Branch Specifier

*/main

👉 Tells Jenkins to monitor main branch

🔹 6. Trigger Setup

✔ Select:
GitHub hook trigger for GITScm polling

👉 Meaning:

- When code is pushed → GitHub sends signal → Jenkins starts build automatically

---

🔹 7. Build Steps

✔ Execute Shell:

```
echo "Build Started"

python3 add.py
```

echo "Build Finished"

👉 What happens:

- Prints start message
- Runs your Python file
- Prints end message

🔹 8. Webhook Setup (Important)**

Step:

- Go to GitHub → Settings → Webhooks
- Add Payload URL:

http://<your-ip>:8080/github-webhook/

👉 You used:

ip addr

to find IP (correct 👍)

Content Type:

application/json

👉 This sends data properly to Jenkins

🔹 9. Build Now

- Click Build Now
- Open Console Output

👉 Shows:

- Build logs
- Output of your Python program

🔥 Overall Flow (Very Important for Viva)

👉
GitHub (code push) → Webhook → Jenkins → Build → Run Python code

---

🎯 Key Viva Questions

❓ What is Jenkins?

→ Automation tool for building, testing, deploying code

❓ What is CI?

→ Continuous Integration (automatic testing when code changes)

❓ What is webhook?

→ Message sent from GitHub to Jenkins to trigger build

❓ What is build?

→ Process of running and testing code

❓ Why use Jenkins?

→ Removes manual work, saves time, avoids errors

❓ What is freestyle project?

→ Basic Jenkins job to run scripts

❓ **What is polling?**

→ **Checking repository for changes**

✅ 1. Flask Application (app.py)

```
from flask import Flask, request, jsonify


# Create Flask app
app = Flask(__name__)


# Home route
@app.route('/')
def home():
    return "AI Model Running Successfully!"


# Prediction route
@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['number']   # get number from request
    result = data * 2               # simple logic (AI demo)
    return jsonify({'result': result})


# Run the application
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

🔍 Explanation

- `Flask` → creates web application
- `@app.route('/')` → homepage
- `@app.route('/predict')` → API endpoint
- `request.json` → gets input from user
- `jsonify()` → sends response in JSON format
- `0.0.0.0` → accessible from any system

👉 **Simple meaning:**
This code creates an API where user sends a number and gets result.

✅ 2. requirements.txt

flask

numpy

👉 Used to install dependencies inside container

✅ 3. Dockerfile (Correct Version)

Use official Python image

FROM python:3.9

Set working directory

WORKDIR /app

Copy requirements file

COPY requirements.txt .

Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

```
# Copy all files
COPY . .

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

🔍 Explanation

- `FROM python:3.9` → base image
- `WORKDIR /app` → working folder inside container
- `COPY` → copies files
- `RUN pip install` → installs libraries
- `EXPOSE 5000` → opens port
- `CMD` → runs app

👉 **Simple meaning:**
Dockerfile tells Docker how to run your project.

✅ 4. Build Docker Image

docker build -t vsec6 .

👉 Creates Docker image named `vsec6`

✅ 5. Run Docker Container

docker run -p 5000:5000 vsec6

👉 Runs app inside container

✅ 6. Test in Browser

http://localhost:5000

✅ 7. Test API using curl

```
curl -X POST http://localhost:5000/predict \

-H "Content-Type: application/json" \

-d '{"number": 5}'
```

👉 Output:

```
{"result": 10}
```

🔥 Full Flow (Very Important)

👉
Flask App → Dockerfile → Build Image → Run Container → API Works

---

🎯 Viva Ready Points

- Docker ensures same environment everywhere
- Flask is used to deploy AI model as API
- Container runs application
- Image is blueprint of container

✅ VSEC 7 – Multi-Container AI App using Docker Compose

🎯 Aim

To create and manage a multi-container AI application (backend, database, frontend) using Docker Compose.

🗂️ 1. Folder Structure

```
vsec7/
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── index.html
│   └── Dockerfile
└── docker-compose.yml
```

✅ 2. Backend Code (app.py)

```
from flask import Flask

from pymongo import MongoClient


app = Flask(__name__)


# Connect to MongoDB container

client = MongoClient("mongodb://db:27017/")
```

```
db = client.testdb


@app.route('/')
def home():
    return "AI Model Running Successfully!"


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

✅ 3. requirements.txt

```
Flask
pymongo
```

✅ 4. Backend Dockerfile

```
FROM python:3.9


WORKDIR /app


COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt


COPY app.py .


CMD ["python", "app.py"]
```

✅ 5. Frontend (index.html)

```
<!DOCTYPE html>

<html>

<head>

<title>AI Docker App</title>

</head>

<body>

<h1>AI Docker Application</h1>

</body>

</html>
```

✅ 6. Frontend Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html
```

✅ 7. docker-compose.yml

```
version: '3'


services:

 backend:

   build: ./backend

   ports:
```

```
    - "5000:5000"
  depends_on:
    - db

db:
  image: mongo
  ports:
    - "27017:27017"

frontend:
  build: ./frontend
  ports:
    - "8080:80"
```

▶️ 8. Run Application

docker-compose up --build

🔍 9. Check Running Containers

docker ps

🌐 10. Access Output

- Frontend → http://localhost:8080
- Backend → http://localhost:5000

docker - - version

(check if the docker compose installed or not,if no installation then install it otherwise continue)
Terminal = docker-compose - - version

If unknown command found
Terminal = sudo apt install docker-compose -y
(wait 1-2 min)

🔥 Working Flow (VERY IMPORTANT)

👉
Frontend (HTML via Nginx)
⬇️
Backend (Flask API)
⬇️
Database (MongoDB)

🎯 Viva Explanation (Simple)

👉 This project uses Docker Compose to run multiple services:

- Backend → Flask application
- Database → MongoDB
- Frontend → Nginx server

All containers are defined in one YAML file and run together.

🎯 Important Viva Questions

❓ What is Docker Compose?

→ Tool to run multiple containers using one file

❓ What is a service?

→ Each container (backend, db, frontend)

❓ What is depends_on?

→ Ensures one container starts before another

❓ Why MongoDB used?

→ NoSQL database for storing data

❓ What is Nginx?

→ Web server to serve frontend

❓ Why use Docker Compose?

→ Simplifies multi-container management